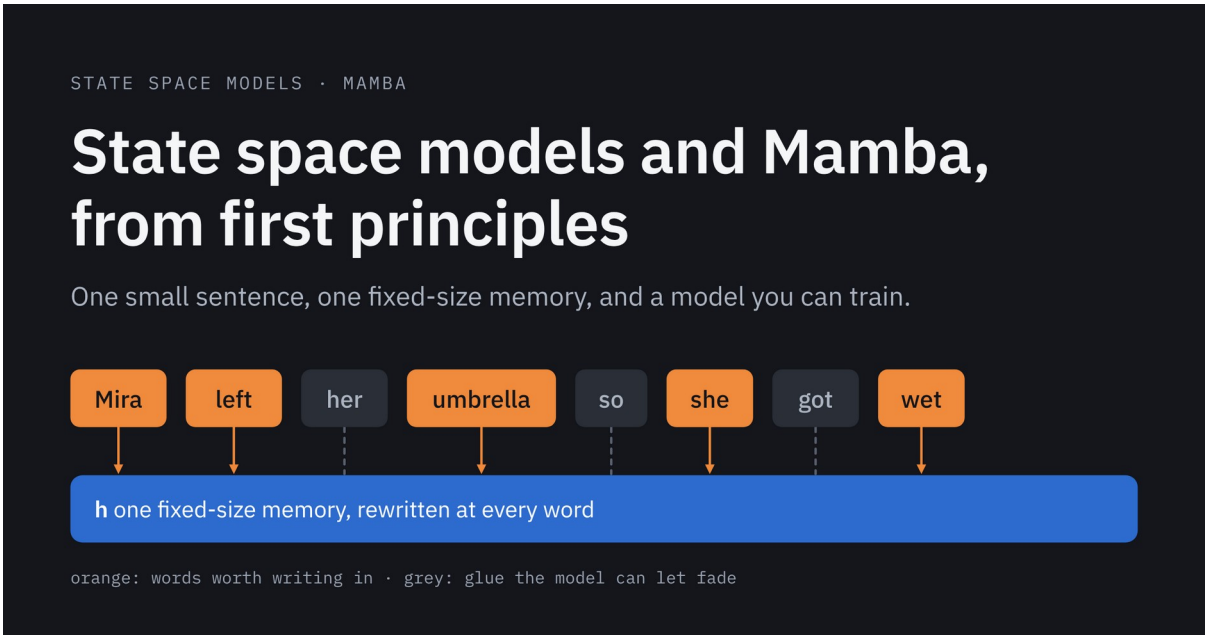
State Space Models and Mamba, from First Principles

One small sentence, one fixed-size memory, and a Mamba you can train and test in plain PyTorch



Cover image (use as the Medium story header).

`image file: fig00-cover.png`

Transformers changed how we model language, but they carry a cost that grows with every token they read. State space models, and Mamba in particular, take a different route: they squeeze everything they have read into a memory of fixed size and decide, word by word, what deserves a place in it.

This post builds that idea from the ground up. We start with a sixty-year-old tool from control engineering, turn it into something that can read text, make it fast to train, make it *selective*, and end with a complete Mamba language model in about 700 lines of plain PyTorch that you can train on a laptop, test, and sample from. Every code snippet below comes from that one script, `mamba_lm.py`.

One sentence to follow all the way through

We will use a single eight-word sentence as our running example:

Mira left her umbrella, so she got wet.

It is short, but it contains the two things that make language modeling hard:

- **A reference that spans words.** When the model reaches *she*, it must still know that *she* is Mira, five words back.
- **A cause that explains a later word.** To predict *wet*, the model needs to remember that the umbrella was left behind, four words earlier.

It also contains words that matter very little. *her* and *so* are glue: a model can let them fade with almost no loss of meaning. Keep that split in mind, content words versus glue, because it is exactly what Mamba learns to exploit.

The problem: carrying the past forward

A sequence is anything that arrives in order: words, stock prices, audio frames, sensor readings. The task is always the same: given everything seen so far, predict what comes next. The hard part is memory. The next word may depend on something said far back, but storing *everything* eventually becomes too slow and too expensive.

FIGURE 1

Three ways to carry the past forward

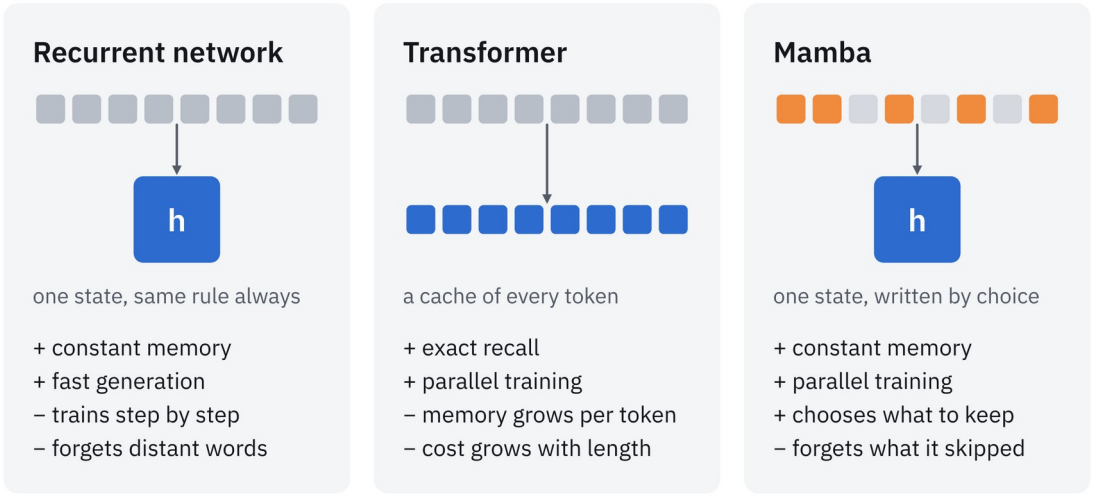


Figure 1. Three ways to carry the past forward. Orange squares are words worth keeping; grey ones are glue.

image file: fig01-three-ways.png

There have been two classic answers, each with a flaw.

Recurrent neural networks (RNNs) keep one running summary, the hidden state, and update it with every new item. Memory stays small and fixed, so generation is fast. But training must happen one step at a time, and information about distant words tends to fade.

Transformers keep every past token and, with attention, compare the current token against all of them. Nothing is forgotten and training parallelizes beautifully. The price: attention cost grows quadratically with sequence length during training, and during generation the cache of past tokens grows with every new token.

Mamba aims for the best of both: a fixed-size state like an RNN, parallel training like a transformer, and a learned decision, token by token, about what to write into that state.

State space models: two equations, four matrices

State space models (SSMs) come from control theory, where they describe physical systems: an electrical circuit, a moving car, the weather. The *state* of a system is the smallest set of numbers that fully describes it right now. For a car that might be position, speed and heading. An SSM predicts how that state evolves as new input arrives.

Three signals are involved:

- **x(t)**, the input arriving at this moment.
- **h(t)**, the latent state: a compressed memory of everything seen so far, never observed directly.
- **y(t)**, the output: the prediction, for us the next word.

Two equations connect them:

```
h'(t) = A·h(t) + B·x(t)        (state equation)
y(t)  = C·h(t) + D·x(t)        (output equation)
```

The first says how the memory changes: h'(t) is its rate of change, driven partly by the state itself (through A) and partly by the new input (through B). The second says how the memory is turned into an answer (through C), plus a shortcut straight from the input (D).

FIGURE 2

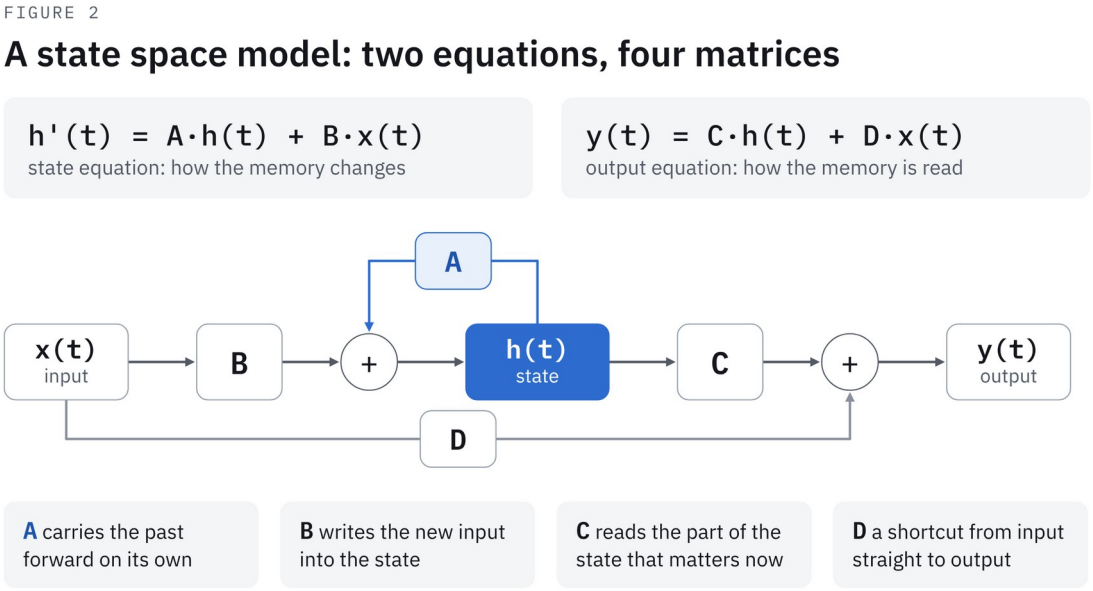


Figure 2. A state space model. A feeds the state back into itself, B writes the input in, C reads the state out, and D bypasses the memory entirely.

image file: fig02-ssm.png

A useful picture is a notebook that gets rewritten at every word. When *umbrella* arrives, the model takes the notebook as it stood after *Mira left her*, keeps what is still useful (that is A's job), writes in the new word (B), and later, when it needs to answer a question, reads the relevant part back out (C).

Matrix	What it controls	In a language model
A	How the state evolves on its own, step to step	Carries the history of the passage forward
B	How a new input is written into the state	Decides what the newest word adds
C	How the state is turned into an output	Decides which stored context shapes the next word
D	A direct path from input to output	A skip connection, often left out of diagrams

In control engineering these matrices are fixed: they describe a system that already exists. In deep learning they are learned weights.

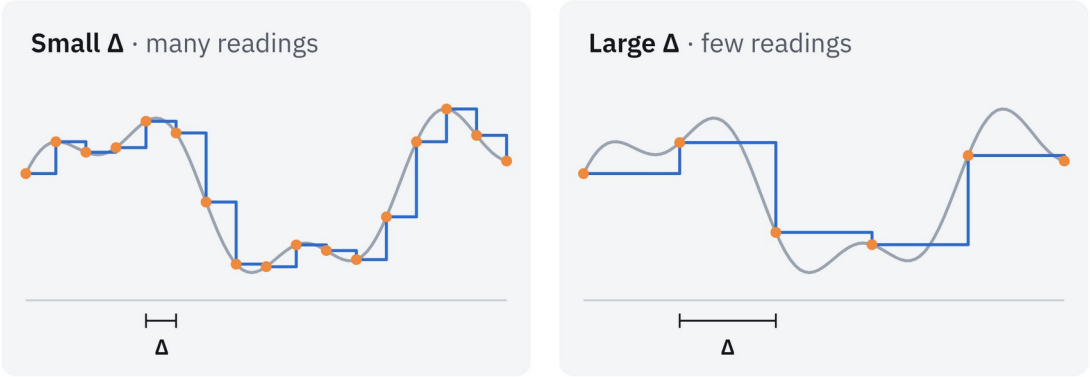
From continuous signals to discrete tokens

SSMs were built for continuous signals, where a value exists at every instant and h'(t) is a true derivative in time. Text is different: it arrives as separate tokens with nothing in between. There is no "halfway between word four and word five".

To bridge the gap we *discretize*: we treat each token as a sample of an underlying continuous signal, taken every Δ units of time. The new parameter Δ, the step size, works like the sampling rate of audio or the frame rate of video. Mamba uses the *zero-order hold* method, which holds each reading constant until the next one.

FIGURE 3

Discretization: the step size Δ sets the resolution



Grey: the continuous signal. **Orange dots**: one reading every Δ. **Blue steps**: each reading held until the next (zero-order hold). Text has no “between”, so Mamba treats each token as one step of size Δ.

Figure 3. The step size Δ sets the resolution. A small Δ takes many readings; a large Δ takes few and holds each one for longer.

image file: fig03-discretization.png

Discretizing turns the two equations into a recurrence over whole steps:

$$h_t = \bar{A} \cdot h_(t\text{-}1) + \bar{B} \cdot x_t$$

$$y_t = C \cdot h_t$$

The bars mark the discretized matrices. Zero-order hold gives $\bar{A} = \exp(\Delta \cdot A)$. For $\bar{B}$ the official Mamba code uses the simpler first-order form $\bar{B} = \Delta \cdot B$, and so does ours. Remember Δ: it will become the key to selectivity.

Written this way, an SSM is literally a recurrent network: one state in, one state out, token by token. Here is that recurrence in code, exactly as the equation reads:

```
# 01_sequential_scan.py  (from mamba_lm.py, docstrings trimmed)
def sequential_scan(a, b):
    h, hs = torch.zeros_like(b[:, 0]), []
    for a_t, b_t in zip(a.unbind(1), b.unbind(1)):  # unbind, not a[:, t]: its backward is one stack, not L
copies
        h = a_t * h + b_t
        hs.append(h)
    return torch.stack(hs, dim=1)
```

One state in, one state out. Which means it inherits both of an RNN's problems: training is sequential, and long-range memory is weak.

S4: one model, two ways to compute it

The Structured State Space sequence model, **S4** (Gu, Goel and Ré, 2021), fixed both problems.

Problem 1, slow training. Each step needs the previous state, so a long sequence has to be processed in order. But a discretized SSM uses only multiplication and addition, so the recurrence can be *unrolled*: every output is a weighted sum of the inputs, with weights built from $\bar{A}$, $\bar{B}$ and C. Those weights form a single convolution kernel K, which can be applied to the whole sequence at once and computed efficiently with the fast Fourier transform.

FIGURE 4

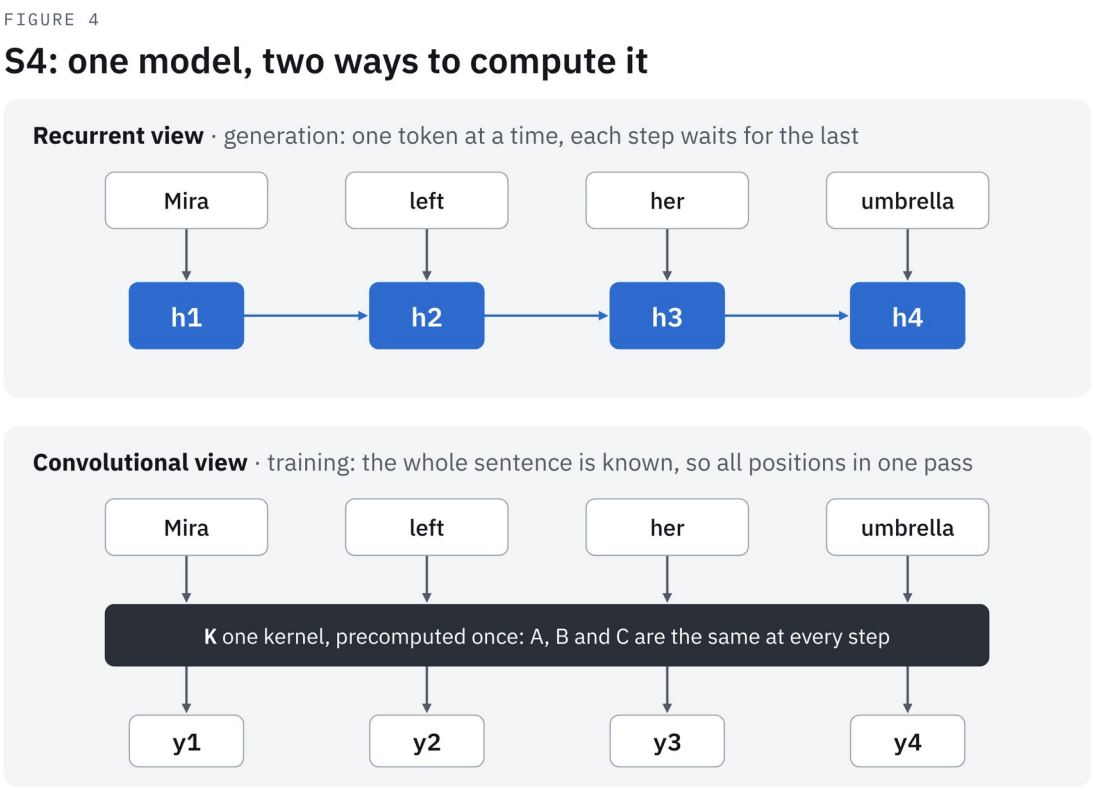


Figure 4. The same model computed two ways: as a recurrence when generating, and as one convolution when training.

image file: fig04-s4-views.png

The catch is that the kernel needs the whole input in advance. That is impossible during generation but exactly the situation during training. So S4 trains as a convolution (like a CNN) and generates as a recurrence (like an RNN), with the same parameters.

Problem 2, weak long-range memory. Instead of initializing A at random, S4 initializes it with **HiPPO** (High-order Polynomial Projection Operators), a matrix derived from orthogonal polynomials that gives the state a principled way to compress a long history. The S4 paper reports that this change alone lifted sequential MNIST accuracy from about 60% to 98%. Later variants (DSS, S4D, S5, Mamba) use simpler diagonal versions of the same idea. Our code uses the simple diagonal "S4D-real" initialization, $A = -1, -2, \ldots, -N$ per channel.

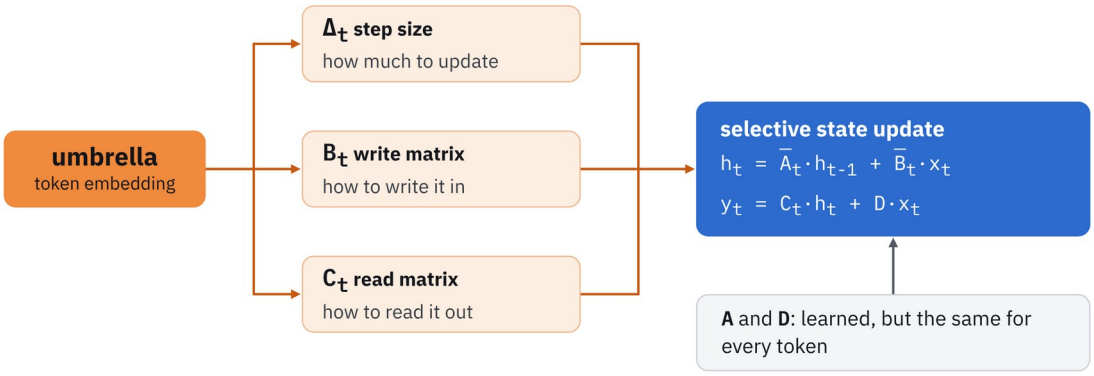
Selectivity: the idea that makes Mamba work

S4 has one remaining limitation, and it is a big one for language. Its matrices are the same at every step: it is *linear time-invariant*. It writes *her* into memory with exactly the same force as *umbrella*. With a fixed-size notebook, that wastes space on glue and risks overwriting the one fact the model will need when it reaches *wet*.

Attention does not have this problem: its weights depend on the content. Mamba (Gu and Dao, 2023) brings that content dependence into the SSM with a **selection mechanism**: Δ, B and C are no longer fixed, they are computed from each token.

FIGURE 5

Selection: each token chooses its own Δ, B and C



Three small linear layers turn the token into its own Δ, B and C. “umbrella” can take a large Δ and write itself firmly into the state; “her” can take a small Δ and barely touch it. A stays fixed: its job is to carry the past, while B and C decide what goes in and what comes out.

Figure 5. Selection. Small linear layers turn each token into its own Δ, B and C. A and D stay the same for every token.

image file: fig05-selection.png

Each input-dependent parameter has a clear job:

- **Δ_t, how much to update.** A large Δ lets the token change the state a lot and speeds up forgetting of older content. A small Δ lets the token pass by

almost without trace. This is how *umbrella* can be written in firmly while *her* barely registers.

- **B_t, how to write it in.** It decides which parts of the state the token goes into.
- **C_t, how to read it out.** It decides which parts of the stored context shape the current output.

A is not made input-dependent: its job remains carrying the whole past forward, while B and C choose what to use. The authors call this model **S6**: S4 with a selection mechanism, computed with a scan. In code, the whole selective SSM is a handful of lines:

```
# 02_selective_scan.py  (from mamba_lm.py, docstrings trimmed)
```

```
def selective_scan(x, delta, A, B, C, D, mode="parallel"):
    with _no_autocast(x):
        x, delta, B, C = x.float(), delta.float(), B.float(), C.float()
        # Discretize (Figure 3). Zero-order hold gives A_bar = exp(delta * A). For B the official
        # implementation uses the simpler first-order form B_bar = delta * B, and so do we.
        A_bar = torch.exp(delta.unsqueeze(-1) * A)  # (batch, L, d, n)
        Bx_bar = (delta * x).unsqueeze(-1) * B.unsqueeze(2)  # (batch, L, d, n)
        h = (parallel_scan if mode == "parallel" else sequential_scan)(A_bar, Bx_bar)
        y = torch.einsum("bldn,bln->bld", h, C)  # y_t = C_t h_t
        return y + x * D  #     + D x_t
```

Every channel has its own small state of N numbers (N = 16 in Mamba-1), so the state at one position is a d × N matrix.

The parallel scan: getting fast training back

Selectivity has a cost. Once Δ, B and C change with every token, there is no longer one fixed kernel, so S4's convolution trick is gone. Are we back to slow, step-by-step training?

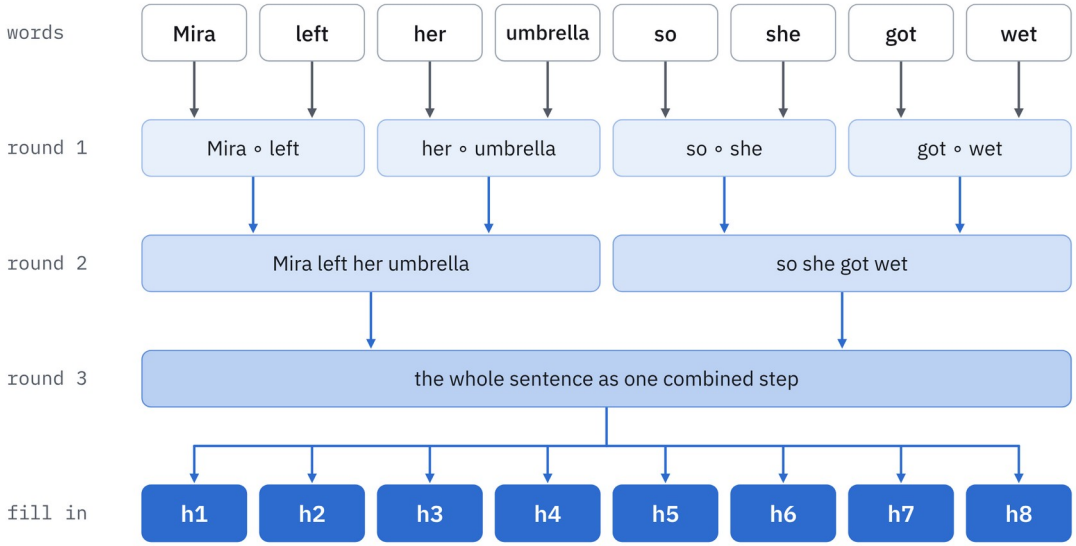
No, thanks to one observation. Each step is just "multiply, then add": h_t = a_t·h_(t-1) + b_t. Two neighbouring steps therefore combine into one step of the same form:

```
h_(2i+1) = (a_(2i+1)·a_(2i))·h_(2i-1) + (a_(2i+1)·b_(2i) + b_(2i+1))
```

That combining rule is associative, so we can group the work however we like. Fuse every pair, then pairs of pairs, and so on, then fill in the state after every word. This is a **parallel prefix scan**.

FIGURE 6

The parallel scan: eight words, three rounds



Each step is “multiply, then add”, so two neighbouring steps fuse into one step of the same form, and the grouping doesn’t change the answer. Fuse pairs, then pairs of pairs, then fill in the state after every word: 8 words take 3 rounds, 1,000 take 10.

Figure 6. The parallel scan on our sentence. Eight words take three rounds instead of eight steps; a thousand take ten.

image file: fig06-parallel-scan.png

Our implementation is a short recursive function that mirrors the figure:

```
# 03_parallel_scan.py  (from mamba_lm.py, docstrings trimmed)
def parallel_scan(a, b):
    L = b.size(1)
    if L == 1:
        return b
    if L % 2:  # odd length: append an identity step (a=1, b=0) so that every token has a partner
        a = torch.cat([a, torch.ones_like(a[:, :1])], dim=1)
        b = torch.cat([b, torch.zeros_like(b[:, :1])], dim=1)
    (a_even, a_odd), (b_even, b_odd) = a.unflatten(1, (-1, 2)).unbind(2), b.unflatten(1, (-1, 2)).unbind(2)
    h_odd = parallel_scan(a_odd * a_even, a_odd * b_even + b_odd)  # the state after each pair
    h_before = torch.cat([torch.zeros_like(h_odd[:, :1]), h_odd[:, :-1]], dim=1)
    h_even = a_even * h_before + b_even  # the state after the first token of each pair
    return torch.stack([h_even, h_odd], dim=2).flatten(1, 2)[:, :L]
```

The official Mamba code goes further with a hardware-aware CUDA kernel that keeps the state in fast on-chip memory, in the spirit of FlashAttention. Our version keeps everything in plain PyTorch so that every line can be read.

The Mamba block

The selective SSM sits inside a Mamba block much like attention sits inside a transformer block. Stacking these blocks gives the full model.

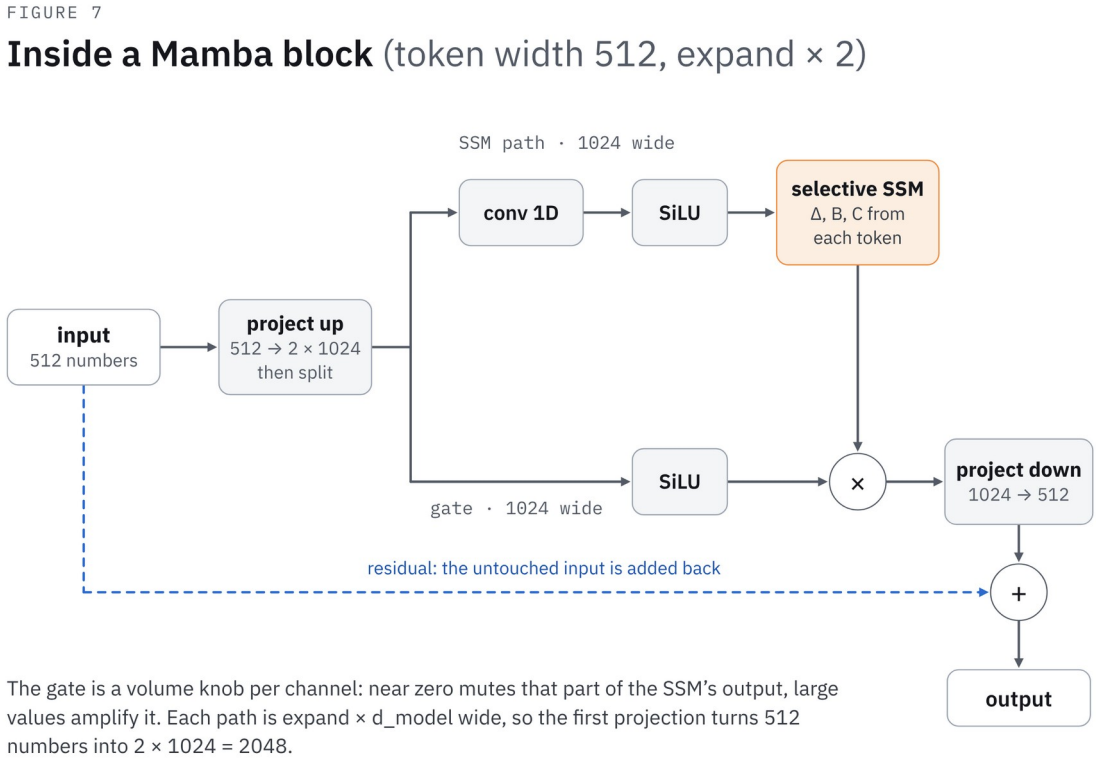


Figure 7. Inside a Mamba block, for tokens of width 512 with an expansion factor of 2.

image file: fig07-mamba-block.png

Step by step, for a token vector of width 512:

1. **Expand and split.** One linear layer produces two vectors, each expand × d_model = 1024 wide: one for the SSM path (x) and one for a gate (z). So the projection outputs 2 × 1024 = 2048 numbers.
2. **Convolve.** A short causal 1-D convolution (width 4) on the SSM path picks up local patterns such as neighbouring-word pairs, leaving the SSM free to focus on long-range structure.
3. **Activate.** A SiLU nonlinearity.
4. **Select and scan.** Compute Δ, B and C from the token, then run the selective scan.
5. **Gate.** Multiply element-wise by SiLU(z). Each element acts like a volume knob: near zero mutes that channel, large values amplify it.
6. **Project down** back to width 512.
7. **Add** the result to the untouched input (a residual connection) and pass it to the next block.

In code, those steps are one short method:

```
# 04_mamba_block_forward.py  (from mamba_lm.py, docstrings trimmed)
def forward(self, u):  # u: (batch, L, d_model)
    L = u.size(1)
```

```
    x, z = self.in_proj(u).chunk(2, dim=-1)  # 01-02
    x = self.conv1d(x.transpose(1, 2))[..., :L].transpose(1, 2)  # 03 (the first L outputs are the causal
ones)
    x = F.silu(x)  # 04
    delta, B, C = self.x_proj(x).split([self.dt_rank, self.d_state, self.d_state], dim=-1)
    delta = F.softplus(self.dt_proj(delta))  # delta_t > 0
    A = -torch.exp(self.A_log.float())
    mode = self.scan if self.scan != "auto" else ("sequential" if u.device.type == "cpu" else "parallel")
    y = selective_scan(x, delta, A, B, C, self.D, mode)  # 05 Select and scan
    y = y * F.silu(z)  # 06 Gate: the z path's SiLU is the volume knob
    return self.out_proj(y)  # 07
```

For generation the block also has a `step` method that reads a single token, updates a fixed-size convolution window and the SSM state, and returns the output. This is the notebook from earlier, literally:

```
# 05_mamba_block_step.py  (from mamba_lm.py, docstrings trimmed)
def step(self, u, cache):  # u: (batch, d_model), a single token
    conv_window, h = cache
    x, z = self.in_proj(u).chunk(2, dim=-1)
    conv_window = torch.cat([conv_window[..., 1:], x.unsqueeze(-1)], dim=-1)
    x = F.silu((conv_window * self.conv1d.weight.squeeze(1)).sum(-1) + self.conv1d.bias)
    delta, B, C = self.x_proj(x).split([self.dt_rank, self.d_state, self.d_state], dim=-1)
    delta = F.softplus(self.dt_proj(delta)).unsqueeze(-1)  # (batch, d, 1)
    A = -torch.exp(self.A_log.float())
    h = torch.exp(delta * A) * h + delta * x.unsqueeze(-1) * B.unsqueeze(1)  # h_t = A_bar h_{t-1} + B_bar
x_t
    y = torch.einsum("bdn,bn->bd", h, C) + self.D * x  # y_t = C_t h_t + D x_t
    return self.out_proj(y * F.silu(z)), (conv_window, h)
```

Mamba-2 and state space duality

A year later, Tri Dao and Albert Gu published "Transformers are SSMs", introducing **Mamba-2** and the idea of **state space duality (SSD)**. Two changes matter most:

- **A is a single number per head** (times the identity), rather than a value per channel and state. That small restriction lets the scan be rewritten almost entirely as matrix multiplications, which GPUs are extremely good at, and makes much larger states affordable.
- **A parallel block.** z, x, B, C and Δ all come out of one projection at the entrance, rather than being produced deeper inside the SSM path.

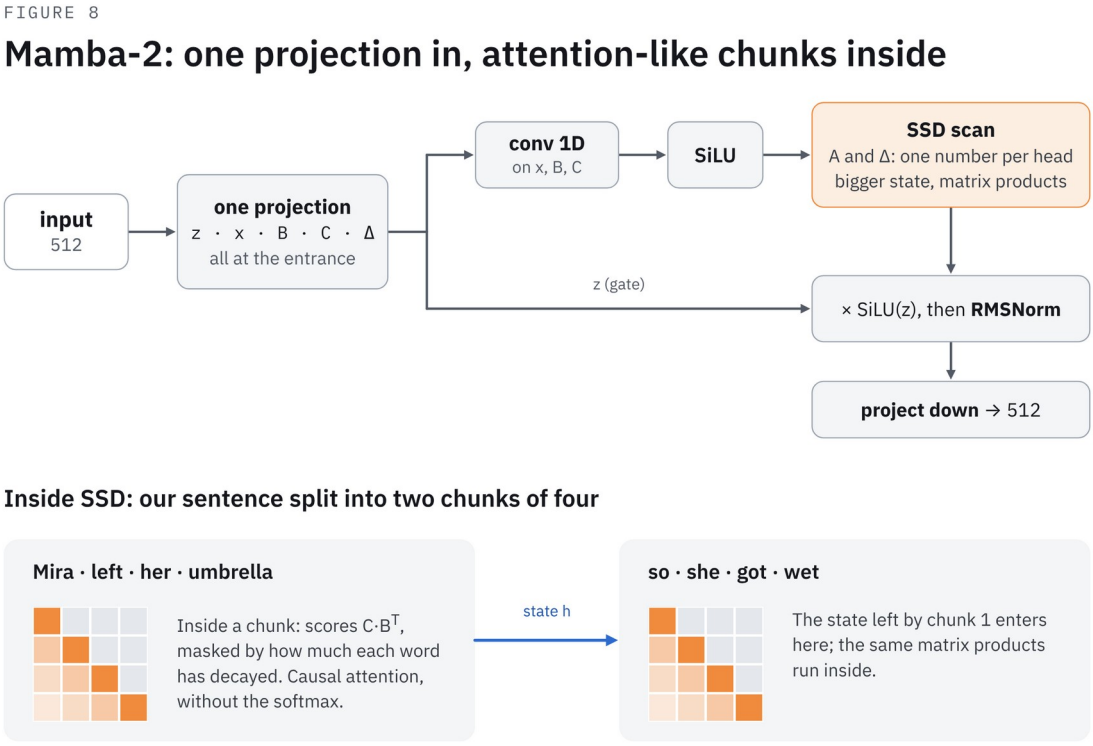


Figure 8. Mamba-2: one projection in, attention-like chunks inside.

image file: fig08-mamba2.png

The duality is the striking part. Split the sequence into chunks. *Inside* a chunk, the SSM output is computed exactly like causal attention: C plays the role of the queries, B the keys, x the values, and a decay matrix replaces the softmax. *Between* chunks, it is a recurrence with one step per chunk. Here is the whole algorithm, adapted from the reference implementation:

```
# 06_ssd.py  (from mamba_lm.py, docstrings trimmed)
def ssd(x, log_decay, B, C, chunk):
    b, L, nh, p = x.shape
    pad = -L % chunk
    if pad:  # pad the end; later positions never affect earlier ones
        x, log_decay, B, C = (F.pad(t, (0, 0) * (t.dim() - 2) + (0, pad)) for t in (x, log_decay, B, C))
    c = x.size(1) // chunk
    x = x.reshape(b, c, chunk, nh, p)
    B, C = B.reshape(b, c, chunk, -1), C.reshape(b, c, chunk, -1)
    a = log_decay.reshape(b, c, chunk, nh).permute(0, 3, 1, 2)  # (batch, heads, chunks, chunk)
    a_cum = torch.cumsum(a, dim=-1)

    # 1. within each chunk: masked attention, scores = C . B, mask = decay matrix
    scores = torch.einsum("bcln,bcsn->bcls", C, B)
    y_diag = torch.einsum("bcls,bhcls,bcshp->bclhp", scores, torch.exp(segsum(a)), x)
    # 2. the state each chunk would leave behind if it had started from zero
    states = torch.einsum("bcln,bhcl,bclhp->bchpn", B, torch.exp(a_cum[..., -1:] - a_cum), x)
    # 3. pass states from chunk to chunk: the recurrence, one step per chunk
    states = torch.cat([torch.zeros_like(states[:, :1]), states], dim=1)
    chunk_decay = torch.exp(segsum(F.pad(a_cum[..., -1], (1, 0))))
    states = torch.einsum("bhzc,bchpn->bzhpn", chunk_decay, states)[:, :-1]  # the state entering each
chunk
    # 4. what each position reads out of the state that entered its chunk
    y_off = torch.einsum("bcln,bchpn,bhcl->bclhp", C, states, torch.exp(a_cum))
    return (y_diag + y_off).reshape(b, c * chunk, nh, p)[:, :L]
```

The Mamba-2 paper reports that its models matched or beat same-size Mamba and transformer models. In our own small run below, Mamba-2 reached the same loss as Mamba-1 in about half the time on a CPU.

Mamba versus transformers

The two architectures make opposite bets about memory.

FIGURE 9

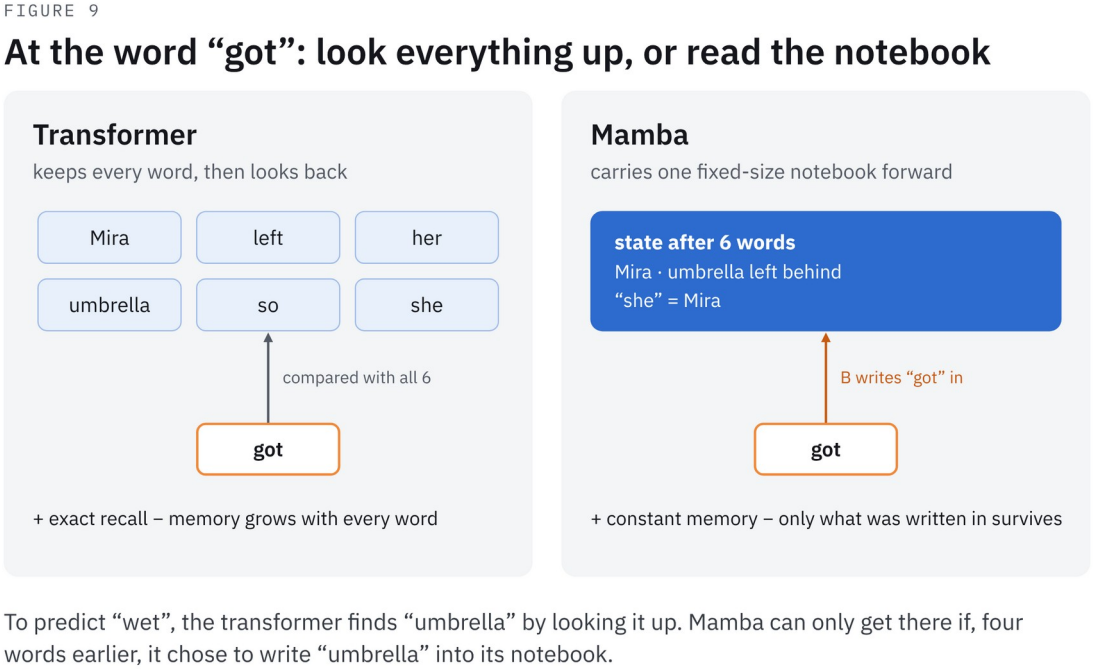


Figure 9. At the word "got": the transformer looks every earlier word up, while Mamba reads its fixed-size notebook.

image file: fig09-lookup-vs-state.png

A transformer keeps every earlier token, so when it needs *umbrella* to predict *wet* it can simply look it up. The cost is a cache that grows with every token. Mamba carries one fixed-size state, so nothing is looked up: it can only predict *wet* if, four words earlier, it chose to write *umbrella* into its notebook.

To make the memory difference concrete, take a model the size of the one we train below: 4 layers, width 128, float32. A transformer of that size stores keys and values for every past token, which is $2 \times 4 \times 128 \times 4$ bytes, about 4 KB per token. Our Mamba carries an 80 KB state no matter how long the text is.

FIGURE 10

Memory carried while generating, as the text grows

Same size as the model trained later (4 layers, d_model 128, float32). Both axes logarithmic.

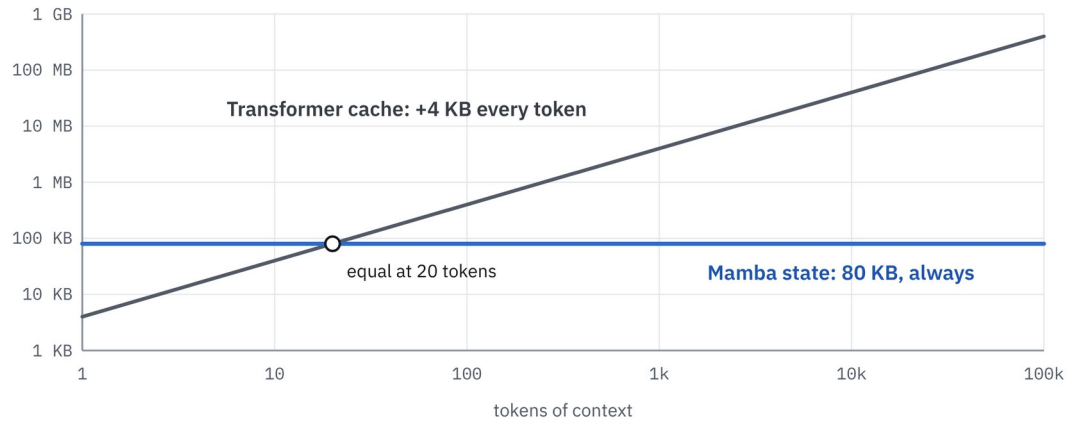


Figure 10. Memory carried while generating. The transformer's cache passes Mamba's fixed state at 20 tokens and keeps growing.

image file: fig10-memory.png

Dimension	Transformer	Mamba
Training cost	Quadratic in sequence length	Linear in sequence length
Generation memory	Grows with every token	Constant, a fixed-size state
Throughput	Baseline	About 5× higher in the Mamba paper
Exact recall and copying	Strong	Weaker: limited to what it chose to keep

The two strengths are not mutually exclusive. **Hybrid models** stack mostly cheap Mamba layers with a few attention layers to restore precise recall. The Mamba-2 paper suggested hybrids could beat either architecture alone, and later work from NVIDIA supported that. IBM's Bamba model and its Granite 4.0 family follow the same pattern.

Building it: a Mamba you can train and test

Everything above is implemented in one file, `mamba_lm.py`, which depends only on PyTorch and runs on CPU, CUDA or Apple MPS. It contains:

- a character-level tokenizer (so any UTF-8 text works, Bangla included);
- the selective scan, with both the sequential and the parallel form;

- the Mamba block and, behind a flag, the Mamba-2 block;
- a language model, a training loop and constant-memory generation;
- a built-in self-test.

The language model

The model is an embedding, a stack of residual blocks (normalize, Mamba block, add back), a final normalization and a projection to the vocabulary. Generation reads the prompt one token at a time and then samples, carrying only the per-layer state:

```
# 07_language_model.py  (from mamba_lm.py, docstrings trimmed)
class ResidualBlock(nn.Module):

    def __init__(self, cfg):
        super().__init__()
        self.norm = RMSNorm(cfg.d_model)
        self.mixer = MambaBlock(cfg) if cfg.arch == "mamba1" else Mamba2Block(cfg)

    def forward(self, x):
        return x + self.mixer(self.norm(x))

    def step(self, x, cache):
        y, cache = self.mixer.step(self.norm(x), cache)
        return x + y, cache



def forward(self, idx, targets=None):
    x = self.embedding(idx)
    for layer in self.layers:
        if self.grad_checkpoint and self.training:  # recompute in backward instead of storing
            x = checkpoint(layer, x, use_reentrant=False)
        else:
            x = layer(x)
    logits = self.lm_head(self.norm_f(x))
    if targets is None:
        return logits
    return logits, F.cross_entropy(logits.float().flatten(0, 1), targets.flatten())

@torch.no_grad()
def generate(self, prompt_ids, max_new_tokens, temperature=1.0, top_k=0):
    device = self.embedding.weight.device
    cache = self.init_cache(1)
    for t in prompt_ids:  # read the prompt like the deck's notebook: one token at a time
        logits, cache = self.step(torch.tensor([t], device=device), cache)
    for _ in range(max_new_tokens):
        if temperature <= 0:
            next_id = logits.argmax(-1)
        else:
            logits = logits / temperature
            if top_k:
                kth = torch.topk(logits, min(top_k, logits.size(-1))).values[..., -1:]
                logits = logits.masked_fill(logits < kth, float("-inf"))
            next_id = torch.multinomial(F.softmax(logits, dim=-1), 1)[:, 0]
        yield next_id.item()
        logits, cache = self.step(next_id, cache)
```

Training

Training is a standard next-character language-modeling loop: random windows of text, cross-entropy loss, AdamW with warmup and a cosine schedule, and gradient clipping. On CUDA the linear layers run in bfloat16, while the scan itself stays in float32, as in the official kernels.

```
# 08_training_step.py  (from mamba_lm.py, docstrings trimmed)
```

```
for step in range(1, args.steps + 1):
    lr = lr_at(step - 1)
    for g in opt.param_groups:
        g["lr"] = lr
    x, y = get_batch(train_data, args.batch_size, args.seq_len, device)
    with amp():
        _, loss = model(x, y)
    opt.zero_grad(set_to_none=True)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
    opt.step()
```

Running it

Four commands cover the whole lifecycle:

```
# 10_run.sh
pip install torch


# 1. check the maths (about ten seconds)
python mamba_lm.py test


# 2. train: downloads Tiny Shakespeare, saves mamba.pt
python mamba_lm.py train


#    the same with the Mamba-2 block, or on your own UTF-8 text
python mamba_lm.py train --arch mamba2
python mamba_lm.py train --data my_corpus.txt


# 3. generate from the checkpoint
python mamba_lm.py generate --prompt "ROMEO:" --max-new 300
```

Testing that the maths is right

A Mamba implementation can be subtly wrong and still train, so the script checks itself. The most important test reads the same random sequence two ways: all at once (the training path) and one token at a time with the fixed-size state (the generation path). If the scan, the convolution window or the discretization were wrong, the two would disagree.

```
# 09_self_test.py  (from mamba_lm.py, docstrings trimmed)
# 2. Reading a sequence all at once == reading it one token at a time with a fixed-size state.
for arch in ("mamba1", "mamba2"):
    cfg = MambaConfig(vocab_size=13, arch=arch, d_model=32, n_layers=2, headdim=16, chunk_size=8,
scan="parallel")
    model = MambaLM(cfg).eval()
    idx = torch.randint(0, 13, (2, 37))  # 37 is deliberately not a multiple of the chunk size
    with torch.no_grad():
        full = model(idx)
        cache, steps = model.init_cache(2), []
        for t in range(idx.size(1)):
            logits, cache = model.step(idx[:, t], cache)
            steps.append(logits)
        stepwise = torch.stack(steps, dim=1)
        if arch == "mamba1":
            for layer in model.layers:
                layer.mixer.scan = "sequential"
            assert torch.allclose(model(idx), full, atol=1e-5), "mamba1: sequential and parallel scans
differ"
    err = (full - stepwise).abs().max().item()
    assert err < 1e-4, f"{arch}: forward and step-by-step differ by {err:.2e}"
    print(f"ok  {arch}: whole-sequence forward == token-by-token recurrence (max difference {err:.1e})")
```

Running `python mamba_lm.py test` prints:

```
ok  parallel scan == sequential recurrence (values and gradients, lengths 1 to 128)
ok  mamba1: whole-sequence forward == token-by-token recurrence (max difference 1.5e-07)
ok  mamba2: whole-sequence forward == token-by-token recurrence (max difference 1.2e-07)
ok  mamba1: learns a repeating sentence (loss 3.29 -> 0.03)
ok  mamba2: learns a repeating sentence (loss 3.26 -> 0.03)
all tests passed
```

I also checked the implementation against an independent one. With identical weights, its outputs match Hugging Face's Mamba and Mamba-2 implementations to within about one part in a million. Because the parameter names follow the official code, the published Mamba-130M checkpoint loads into this model with only a name prefix changed, and it completes "The capital of France is" with "in the city of Paris."

Results

I trained both variants with the default settings on Tiny Shakespeare (about 1.1 million characters) on a 2-core CPU. Each model has about 475,000 parameters.

FIGURE 11

Validation loss while training on Tiny Shakespeare

Two ~475K-parameter models with the script's default settings, measured every 250 steps on a 2-core CPU.

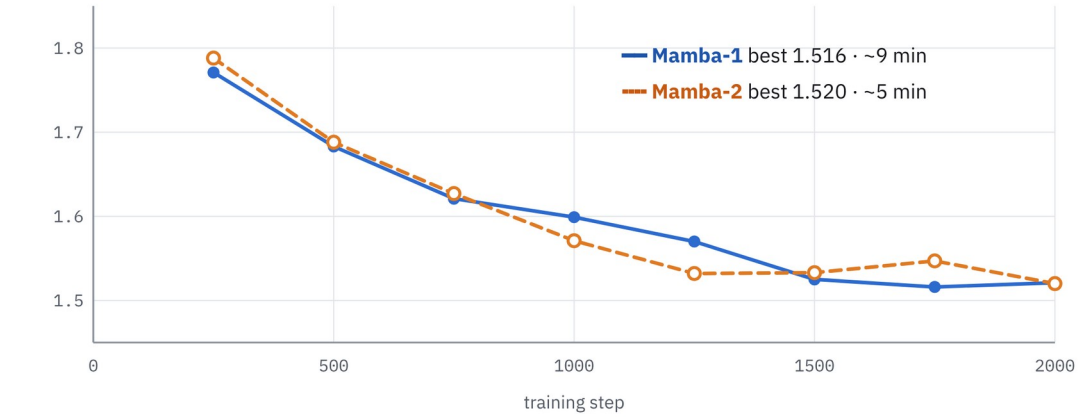


Figure 11. Validation loss every 250 steps. Both variants reach about 1.52 nats per character (2.19 bits); Mamba-2 gets there in roughly half the time.

image file: fig11-training.png

Mamba-1 reached a best validation loss of 1.516 in about 9 minutes, Mamba-2 reached 1.520 in about 5. Here is what the Mamba-1 model writes when prompted with "ROMEO:":

```
ROMEO:
Calls, here is aught, thou met,
Thou shalt way with a dear in helm,
Hearing to the figales, and my parliament
Too well be this cause: but the reversiness!

LADY CAPULET:
She swell i' the view, it rean'd with the duke;
```

Not Shakespeare, but the structure is all there: speaker names, line breaks, archaic forms, punctuation. And while it generates, it carries an 80 KB state, the same after one character as after a million.

Five things to take away

1. A state space model carries one compressed memory forward and reads its answer out of it: two equations, four matrices.
2. Discretization with a step size Δ makes that machinery usable on text, turning it into a recurrence.
3. S4 made it trainable as a convolution and gave it long memory through HiPPO-style initialization.
4. Mamba makes Δ, B and C depend on the current token, so the model chooses what to remember, and recovers fast training with a parallel scan.
5. Constant memory during generation is the payoff. Hybrids that mix Mamba layers with a few attention layers are where much of the current work is.

References

- Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023. arxiv.org/abs/2312.00752
- Tri Dao and Albert Gu. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. 2024. arxiv.org/abs/2405.21060
- Albert Gu, Karan Goel and Christopher Ré. Efficiently Modeling Long Sequences with Structured State Spaces (S4). 2021. arxiv.org/abs/2111.00396
- Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra and Christopher Ré. HiPPO: Recurrent Memory with Optimal Polynomial Projections. 2020. arxiv.org/abs/2008.07669
- Roger Waleffe et al. An Empirical Study of Mamba-based Language Models. NVIDIA, 2024. arxiv.org/abs/2406.07887
- Dave Bergmann. What is a Mamba model? IBM Think, 2025.
- Official implementation: github.com/state-spaces/mamba